

## PROJECT DOCUMENTATION

# SecureAuth AI Copilot

*Behavioral login-risk scoring, step-up verification, and an audit trail. The score is computed by fixed rules. A language model only writes the explanation.*

Document date: 6 October 2026

Repository: Ayush-2308/SecureAuth-AI-Copilot

Status: local FastAPI application with Supabase storage

## 1. What this project is

SecureAuth AI Copilot checks one login attempt against that user's own past logins. It answers a single question: does this attempt look like the user's normal behavior, or should the application allow it, ask for a second proof, or stop it?

The product is a small pipeline, not a chatbot. LangGraph runs six steps in order. Python rules compute a risk score from 0 to 1. Google Gemini may write one to four plain-English reasons for the audit log. Gemini never changes the score. A web page and two HTTP endpoints let a person submit a login and, when required, a one-time code.

The system produces one of four final actions:

- allow - the login may continue.
- challenge\_otp - ask for a 6-digit one-time code.
- challenge\_biometric - ask the client to prompt fingerprint or face unlock again.
- block - deny the login. This happens when there have been more than 10 attempts in the last hour.

## 2. A concrete example

Rahul normally signs in from Mumbai on the phone named phone-rahul. A new attempt arrives from the same city, the same IP, but a device id the history has never seen: tablet-new. There have also been 5 attempts in the last hour.

| Signal                    | Result                                    | Weight added |
|---------------------------|-------------------------------------------|--------------|
| New device                | Yes                                       | +0.30        |
| New location              | No, Mumbai is known                       | +0.00        |
| Unusual hour              | Not enough of a personal hour pattern yet | +0.00        |
| Attempts in the last hour | 5, which is at least 3                    | +0.25        |
| Bad IP reputation         | Not flagged (placeholder)                 | +0.00        |
| Score                     | 0.55, capped at 1.00                      | medium       |

0.55 is at or above the medium threshold of 0.30 and below the high threshold of 0.70, so the level is medium. The decision step maps medium to challenge\_otp. The API confirms that a code was sent. It does not return the digits. If the submitted code's SHA-256 hash matches the stored hash before expiry, the screen says the login can continue.

## 3. Architecture

A browser page posts a login to FastAPI. FastAPI calls run\_pipeline. LangGraph moves one PipelineState through the nodes below. The pattern agent reads recent rows from Supabase. The scoring agent adds weights locally and optionally calls Gemini for reasons. The decision agent chooses the action. Unless the action is block, the verification agent issues a challenge. The store node writes the login, the assessment, an OTP hash when one was issued, and a security-event line.

| Layer         | Responsibility                         | Code                         |
|---------------|----------------------------------------|------------------------------|
| Web UI        | Login form, decision panel, OTP box    | static/index.html            |
| HTTP API      | POST /login-event and POST /verify-otp | main.py                      |
| Orchestration | StateGraph and run_pipeline            | secureauth/graph.py          |
| Behavior      | Compare this login with history        | agents/pattern_agent.py      |
| Score         | Weights, level, LLM reasons            | agents/risk_scoring_agent.py |
| Decision      | Final action, including the hard block | agents/decision_agent.py     |

| Layer        | Responsibility                   | Code                         |
|--------------|----------------------------------|------------------------------|
| Verification | OTP hash, biometric prompt, deny | agents/verification_agent.py |
| Database     | Supabase reads and writes        | db/supabase_client.py        |
| Schema       | Pydantic models                  | schemas/models.py            |
| Settings     | Environment variables            | config.py                    |

## 4. LangGraph pipeline

The graph state is the Pydantic model PipelineState. run\_pipeline creates an event id and starts the state as pending. Each node returns only the fields it changes.

| Order | Node             | What it writes                                             |
|-------|------------------|------------------------------------------------------------|
| 1     | intake           | status = received                                          |
| 2     | extract_features | RiskFeatures from Supabase history                         |
| 3     | score_risk       | RiskAssessment, including a provisional action             |
| 4     | decide           | Replaces recommended_action with the final action          |
| 5     | verify           | verification_result. Skipped when the action is block      |
| 6     | store            | login_history, risk_assessments, otp hash, security_events |

The conditional edge sits after decide. If recommended\_action is block, the next node is store. Every other action goes to verify, then store. The returned status is stored.

## 5. Behavioral features

extract\_features loads up to 20 of the user's most recent login\_history rows when the caller does not pass history. Rows for a different user\_id are ignored. Timestamps may be datetimes or ISO-8601 strings.

| Feature                       | Rule                                                                                                                                       |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| is_new_device                 | True when this device_id has not appeared in history.                                                                                      |
| is_new_location               | True when the current location is present and does not match a previous location, ignoring case. A missing location is not treated as new. |
| time_since_last_login_minutes | Minutes since the latest earlier login. None when there is no earlier login.                                                               |
| login_attempts_last_hour      | Logins in the 60 minutes up to this attempt, including this attempt, without counting this attempt twice.                                  |
| is_unusual_hour               | True only after at least 3 earlier logins, and only when this clock hour has never appeared in them.                                       |
| ip_reputation_flag            | Always false for now. A real reputation lookup is still a TODO.                                                                            |

## 6. Risk metrics

The score is a sum of weights. It is capped at 1.0. These numbers are the project metrics. They are not learned by the model.

| Metric              | Value | When it applies                       |
|---------------------|-------|---------------------------------------|
| New device weight   | 0.30  | is_new_device is true                 |
| New location weight | 0.20  | is_new_location is true               |
| Unusual hour weight | 0.15  | is_unusual_hour is true               |
| High attempt weight | 0.25  | login_attempts_last_hour is 3 or more |
| Bad IP weight       | 0.40  | ip_reputation_flag is true            |
| Score cap           | 1.00  | The sum cannot exceed this            |

| Metric               | Value            | When it applies                                         |
|----------------------|------------------|---------------------------------------------------------|
| Medium threshold     | 0.30             | score $\geq$ 0.30 and $<$ 0.70 is medium. Configurable. |
| High threshold       | 0.70             | score $\geq$ 0.70 is high. Configurable.                |
| Low band             | below 0.30       | Everything under the medium threshold                   |
| Hard attempt block   | more than 10     | Overrides the level and forces block                    |
| Hour-pattern minimum | 3 earlier logins | Before that, the hour is not called unusual             |
| History window       | 20 rows          | Most recent logins for that user                        |
| OTP lifetime         | 5 minutes        | After that, verify_otp returns false                    |
| OTP length           | 6 digits         | Generated with secrets, not the random module           |
| Gemini timeout       | 10 seconds       | If the call fails, rule-written reasons are used        |

Maximum raw sum is  $0.30 + 0.20 + 0.15 + 0.25 + 0.40 = 1.30$ , which is stored as 1.00. A brand-new user with a location is typically 0.50: new device 0.30 plus new location 0.20. That is medium, so the final action is challenge\_otp.

## 7. How the final action is chosen

score\_risk first attaches a provisional action: low becomes allow, medium becomes challenge\_otp, and high becomes block. The decide node then replaces that action. The value stored in Supabase and returned by the API is the decision-agent result, not the provisional one.

| Condition                                  | Final action                    |
|--------------------------------------------|---------------------------------|
| More than 10 attempts in the last hour     | block, even if the score is low |
| Level low, and attempts are 10 or fewer    | allow                           |
| Level medium, and attempts are 10 or fewer | challenge_otp                   |
| Level high, and attempts are 10 or fewer   | challenge_biometric             |

## 8. Verification

This step mirrors the step-up checks used by production banking and fintech apps. allow returns success. challenge\_biometric returns a payload that tells the client to prompt biometric authentication again. block is logged and returns denied, and the graph skips this node entirely. challenge\_otp creates a 6-digit code, stores only SHA-256(code), and sets a 5-minute expiry. The plaintext code is not written to Supabase and is not included in the HTTP response. verify\_otp hashes the submitted code, compares it in constant time, rejects an expired or unknown code, and deletes a matching code so it cannot be reused. The live check uses the in-memory challenge created by this server process.

## 9. Supabase tables

Create these tables by running secureauth/db/migrations.sql in the Supabase SQL editor. Each table also has a uuid primary key generated by the database.

| Table            | Written by         | Important columns                                                                        |
|------------------|--------------------|------------------------------------------------------------------------------------------|
| login_history    | save_assessment    | user_id, timestamp, ip_address, device_id, location, login_method                        |
| risk_assessments | save_assessment    | event_id, user_id, risk_score, risk_level, recommended_action, reasons jsonb, created_at |
| otp_challenges   | store_otp          | user_id, otp_hash, expires_at, verified                                                  |
| security_events  | log_security_event | event_id, detail, created_at                                                             |

get\_user\_login\_history reads login\_history for one user, newest first, default limit 20. Failed API calls also try to write a security\_events row. The detail names the operation and the exception type. It does not include the one-time code.

## 10. API and user interface

GET / serves static/index.html. The page is a login form and a decision panel.

| Method and path   | Body            | Success response                                                        |
|-------------------|-----------------|-------------------------------------------------------------------------|
| POST /login-event | LoginEvent JSON | assessment, recommended_action, challenge_sent, message. No OTP digits. |
| POST /verify-otp  | user_id and otp | success true or false                                                   |

LoginEvent fields are user\_id, timestamp, ip\_address, device\_id, device\_type, optional location, and login\_method. A first login from a new device and a new place can return risk\_score 0.5, risk\_level medium, recommended\_action challenge\_otp, and challenge\_sent true. If the same device and place are used again and the other signals stay quiet, the score can be 0.00 and the action is allow, which the page shows as "Login can continue." After a matching OTP, the decision panel is replaced with the same sentence.

## 11. Technology used

| Piece                    | Role in this project                                          |
|--------------------------|---------------------------------------------------------------|
| Python 3.11+             | Application language                                          |
| FastAPI                  | HTTP API                                                      |
| Uvicorn                  | Local server                                                  |
| Pydantic                 | LoginEvent, RiskFeatures, RiskAssessment, PipelineState       |
| LangGraph                | StateGraph that orders the six nodes                          |
| langchain-core           | Declared dependency of the agent stack                        |
| Google Gemini            | gemini-3.8-flash, reasons only, provider google               |
| httpx                    | Gemini REST call                                              |
| Supabase                 | Postgres tables behind the data API                           |
| python-dotenv            | Loads the project-root .env file                              |
| hashlib SHA-256 and hmac | OTP storage and constant-time compare                         |
| secrets                  | OTP generation                                                |
| Browser page             | HTML, CSS, and JavaScript in one file. No frontend framework. |

## 12. Configuration

config.py loads .env from the repository root. Do not commit that file. secureauth/.env.example lists the names only.

| Variable              | Purpose                                           | Default                            |
|-----------------------|---------------------------------------------------|------------------------------------|
| SUPABASE_URL          | Project URL                                       | empty, required to run             |
| SUPABASE_KEY          | API key allowed to read and write the four tables | empty, required to run             |
| LLM_API_KEY           | Gemini key for audit reasons                      | empty; rules still score the login |
| LLM_PROVIDER          | Must be google or gemini to call the model        | google in the example file         |
| RISK_THRESHOLD_MEDIUM | Start of the medium band                          | 0.3                                |
| RISK_THRESHOLD_HIGH   | Start of the high band                            | 0.7                                |

## 13. How to run

- From the repository root, install `secureauth/requirements.txt`.
- Copy `secureauth/.env.example` to `.env` and fill in the Supabase and Gemini values.
- Run `secureauth/db/migrations.sql` in the Supabase SQL editor.
- Start the app with: `python -m uvicorn main:app --host 127.0.0.1 --port 8001`
- Open `http://127.0.0.1:8001/` . Port 8000 may already be used by another local app, which is why 8001 is the safe local port.
- Submit a login. If the decision is `challenge_otp`, enter the code for that same user id. Do not submit the login form again before verifying, or a new code replaces the old one.

## 14. Repository map

| Path                                                      | Contents                                     |
|-----------------------------------------------------------|----------------------------------------------|
| <code>main.py</code>                                      | FastAPI application                          |
| <code>static/index.html</code>                            | Login and decision page                      |
| <code>README.md</code>                                    | Short setup and curl examples                |
| <code>secureauth/graph.py</code>                          | Compiled LangGraph pipeline                  |
| <code>secureauth/config.py</code>                         | Environment settings                         |
| <code>secureauth/schemas/models.py</code>                 | Shared data models                           |
| <code>secureauth/agents/</code>                           | Pattern, scoring, decision, and verification |
| <code>secureauth/db/supabase_client.py</code>             | Database helpers                             |
| <code>secureauth/db/migrations.sql</code>                 | Table definitions                            |
| <code>docs/SecureAuth-AI-Copilot-Documentation.pdf</code> | This document                                |

## 15. Limits

- IP reputation is not a live lookup. The flag stays false, so its 0.40 weight is unused until that TODO is replaced.
- Unusual hour needs three earlier logins. A first-time user is not marked unusual just because the clock says 2:15.
- Only the latest 20 logins are read. A longer attack spread outside that window is not fully counted.
- OTP verification on the running API checks the code held in that server process. Restarting the process drops codes that have not yet been verified.
- The HTTP API never returns the OTP. A demo operator cannot read the digits from the JSON response.
- Gemini explains the features. If the key is missing or the call fails, the same features are written as fixed sentences and the score is unchanged.

End of document. The numbers in section 6 and the action table in section 7 match the code as of 6 October 2026.